Network Topology Optimization for Bearing Rigid Multi-Agent Systems via Deep Reinforcement Learning and Graph Neural Networks

Noyan Erdin Kilic
https://github.com/nyanklc/rigidity_rl

31/08/2026

Overview

CONTENTS

Research Questions and Goals

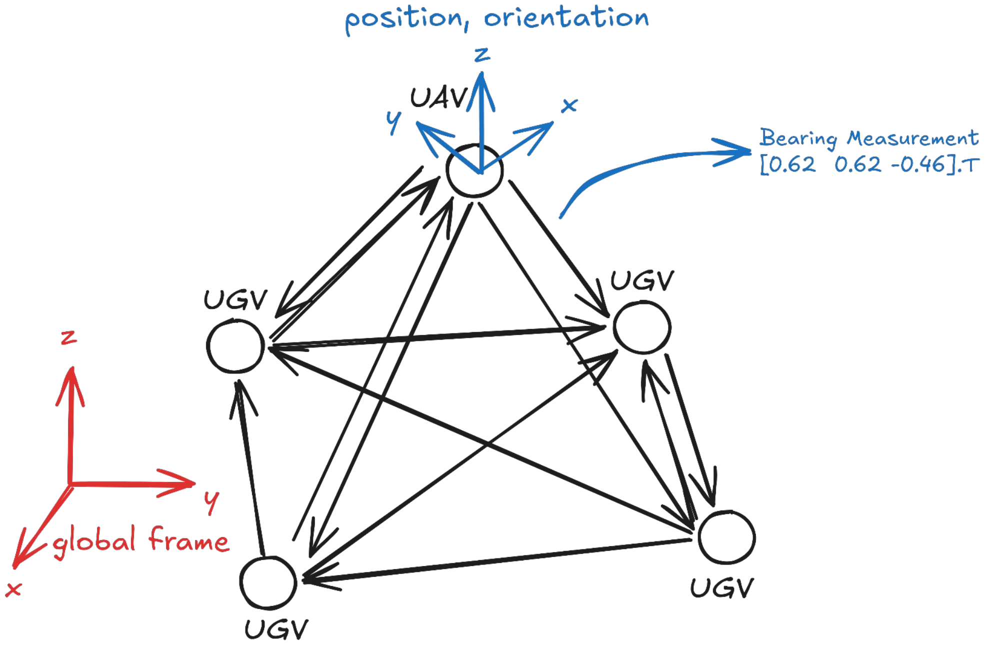
Research Questions

- Is there an optimal graph topology balancing sparsity against structural rigidity, and what characterizes it?
- Can deep reinforcement learning construct such a topology, and what architecture and training procedure does that require?
- Does a learned policy generalize to networks it was not trained on: different sizes, different agent domains, heterogeneous compositions?
- Where does a learned policy earn its place against classical combinatorial heuristics, and where does it not?
- What are the properties of resultant graphs, and why?

Goals

- Study bearing rigidity theory from a topology optimization perspective.
- Develop experimentation environments, policy architectures, and training schemes.
- Define a mathematical objective and train a policy that optimizes it.
- Provide a single policy that generalizes across network types and sizes.
- Evaluate the trained policy, comparing against existing methods and heuristics.
- Study the behavior of the policy, as well as the network topologies it constructs. Draw conclusions based on empirical and theoretical facts.

Bearing Rigidity



Bearing Rigidity

Given all bearings:
[x_0, y_0, z_0, x_1, ...].T

↓

Can we tell the shape of the graph/framework (uniquely)?

(up to a translation/rotation/scale factor)

Bearing Rigidity Matrix

$$\dot{\mathbf{b}}_{\mathcal{G}}(\chi(t)) = \frac{d}{dt}\boxed{\mathbf{b}_{\mathcal{G}}(\chi(t))} = \boxed{\mathbf{B}_{\mathcal{G}}(\chi(t))}\delta.$$

How to check for Infinitesimal Bearing Rigidity?

rank(BRM_graph) == rank(BRM_fullyconnected)

Some other tools at our disposal:

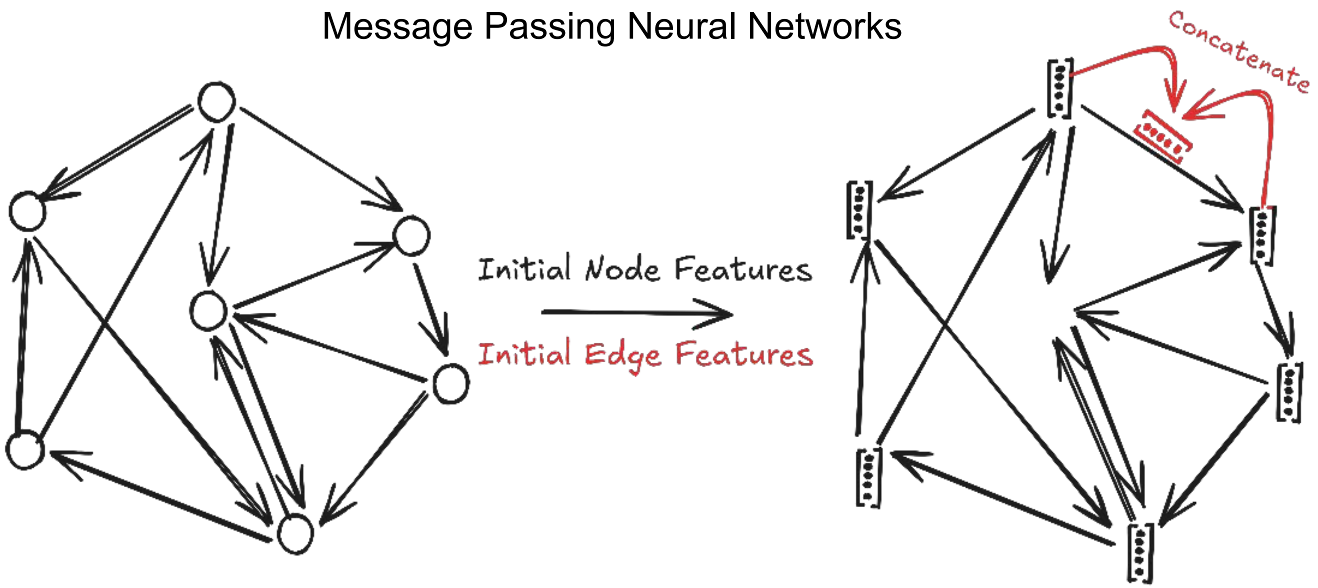
- Minimal Bearing Rigidity: We have a closed form solution to the minimum number of edges needed for a framework to be rigid. This only works on homogeneous networks with domain R^d.

- (Heuristic) Generalized MBR: A greedy per-edge counting of the DOF constraints introduced by the bearing measurements. We can check if these constraints account for all non-trivial DOFs, and if any redundant edges remain. Estimates (on average) a lower number than the true value for heterogeneous networks.

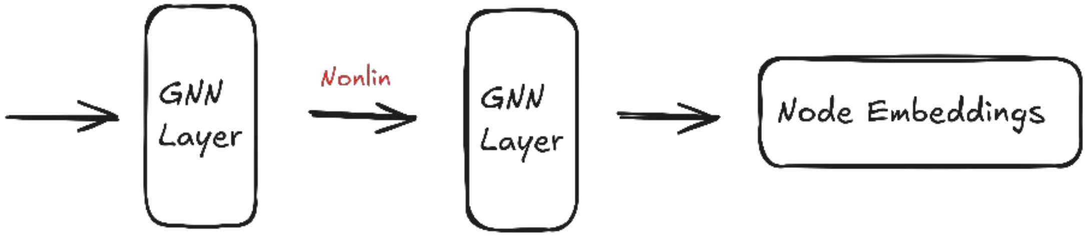
G. Michieletto, A. Cenedese, and D. Zelazo, “A Unified Dissertation on Bearing Rigidity Theory,” IEEE Trans. Control Netw. Syst., vol. 8, no. 4, pp. 1624–1636, Dec. 2021, doi: [10.1109/TCNS.2021.3077712](https://doi.org/10.1109/TCNS.2021.3077712).

Graph Neural Networks

Message Passing Neural Networks



$$\mathbf{x}_i' = \gamma_{\mathbf{\Theta}} \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}(i)} \phi_{\mathbf{\Theta}} \left(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{j,i} \right) \right)$$



Graph Isomorphism Network

$$h_v^{(k)} = \mathrm{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right).$$

Equivariant Graph Neural Network

$$\mathbf{m}_{ij} = \phi_e \left(\mathbf{h}_i^l, \mathbf{h}_j^l, \left\| \mathbf{x}_i^l - \mathbf{x}_j^l \right\|^2, a_{ij} \right)$$

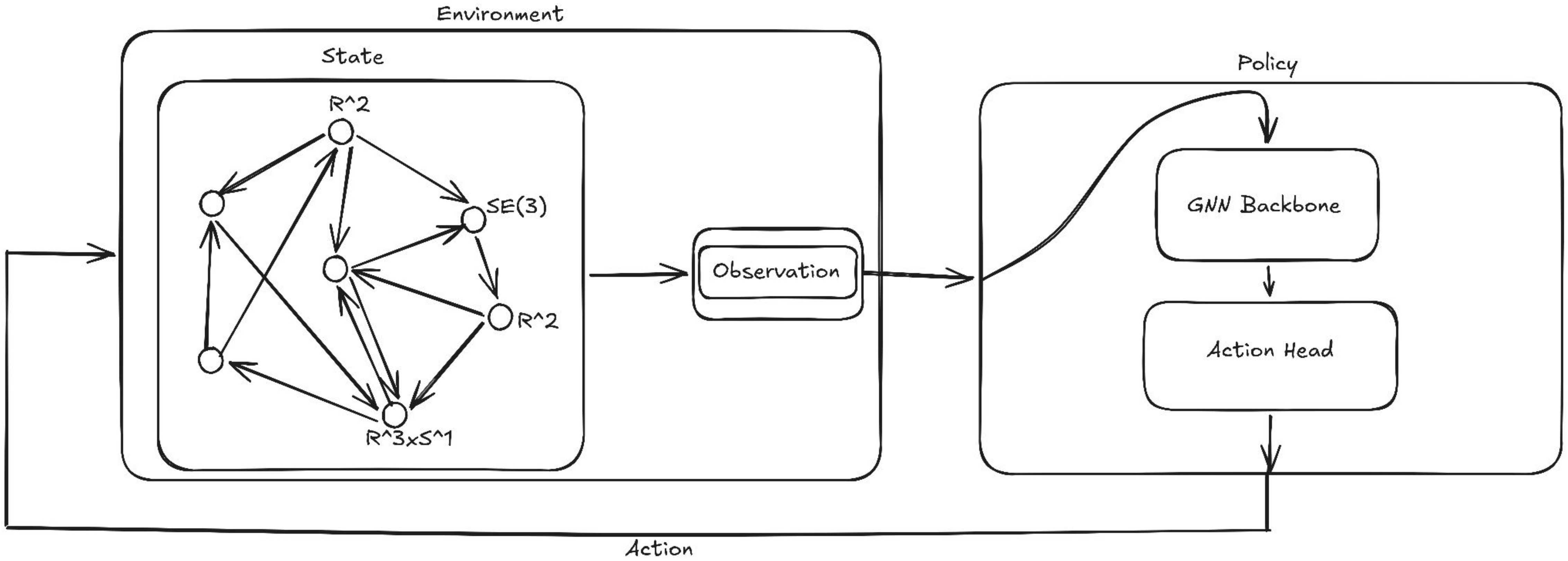
$$\mathbf{x}_i^{l+1} = \mathbf{x}_i^l + C \sum_{j \neq i} \left(\mathbf{x}_i^l - \mathbf{x}_j^l \right) \phi_x \left(\mathbf{m}_{ij} \right)$$

$$\mathbf{m}_i = \sum_{j \neq i} \mathbf{m}_{ij}$$

$$\mathbf{h}_i^{l+1} = \phi_h \left(\mathbf{h}_i^l, \mathbf{m}_i \right)$$

M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” Apr. 25, 2019, *arXiv*: arXiv:1903.02428. doi: 10.48550/arXiv.1903.02428.
K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How Powerful are Graph Neural Networks?,” Feb. 22, 2019, arXiv: arXiv:1810.00826. doi: 10.48550/arXiv.1810.00826.

Approach



Training algorithms: DQN and PPO
Model architectures: GINE and EGNN graph neural networks, MLP action heads
Evaluation:

- Comparison with combinatorial baseline algorithms
- Analysis of resultant topologies

Problems and Assumptions

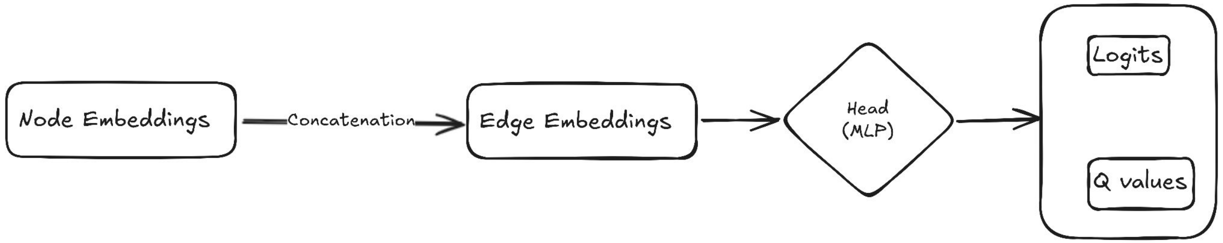
Problems/Considerations:

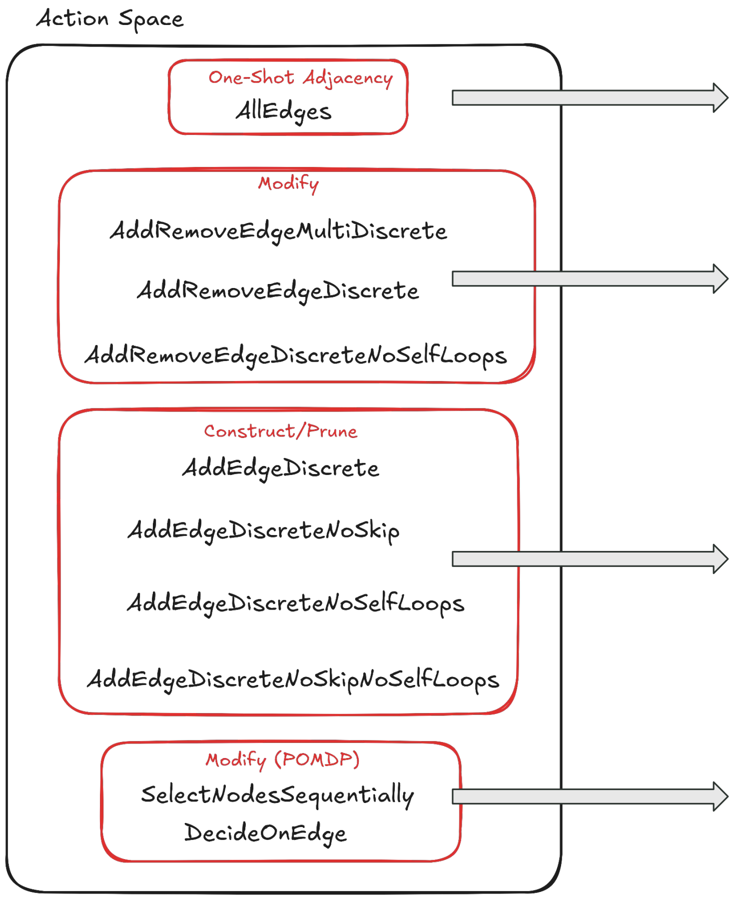
- Absolute positions as node features -> GNNs can’t generalize
- Absolute orientations as node features -> GNNs can’t generalize
- Can’t use bearings as node features -> Node feature size depends on degree
- We would like to use edge features for bearings -> GNNs are generally designed for node features
- We don’t have much information in our state formulation (just poses and bearings) -> Need to derive information
- Action space size change with network size -> Training becomes exponentially more difficult on larger networks

Assumptions:

- The **policy is centralized** and has access to any information available.
- The **policy is informed** with derived rigidity quantities, such as how much rank gain (rank of the bearing rigidity matrix) adding a specific edge would yield etc.

Environment Design - Action Space





$O(n^2)$ vector -> [adj_{00} adj_{01} … adj_{n-1n-1}]

$O(m \sim n^2)$ scalar -> [add_0 add_1 … add_{E-1} $remove_0$ … $remove_{E-1}$ skip/stop]

$O(m \sim n^2)$ scalar -> [add_0 add_1 … add_{E-1} skip/stop]

$O(n)$ / $O(1)$ scalar -> [$select_0$ $select_1$ … $select_{n-1}$ skip/stop] / [yes no skip/stop]

Derived Rigidity Quantities for Observation and Reward

Quantity / Concept	Mathematical Expression	Description
Rigidity Rank	$\mathrm{rank}(B)$	The current rank of the extended bearing-rigidity matrix B.
Infinitesimal Bearing Rigidity	$\mathrm{rank}(B) = r_K$	The condition where the framework is rigid; rank equals the rank of the complete graph r_K.
Unobservable Subspace	$\ker B$	The subspace containing state variations that are invisible to the bearing measurements.
Rigidity Eigenvalue	$\lambda = \min_{\substack{\delta\chi \perp \ker B \\ \|\delta\chi\|=1}} \|B\delta\chi\|^2$	The weakest identifiable direction, acting as the E-optimality criterion for the problem's conditioning.
Softest Mode	$v = \arg\min_{\substack{\delta\chi \perp \ker B \\ \|\delta\chi\|=1}} \|B\delta\chi\|^2$	The specific direction in configuration space with the greatest sensitivity to measurement error.
Error Amplification	$\|\delta\chi\| \le \frac{1}{\sqrt{\lambda}}\|\delta\beta\|$	The worst-case bound describing how measurement error amplifies into shape error.
Estimation Uncertainty	$\mathrm{Cov}(\delta\chi) = \sigma^2(B^\top B)^+$	The first-order reconstruction covariance matrix on the identifiable subspace.
A-optimality	$a_{\mathrm{opt}} = \mathrm{tr}((B^\top B)^+)$	The total mean-squared error criterion across the identifiable subspace.

Quantity / Concept	Mathematical Expression	Description
D-optimality	$d_{\mathrm{opt}} = -\sum_k \log w_k$	The log-volume of the error ellipsoid criterion.
Edge-Addition Rank Gain	$\Delta\,\mathrm{rank}_{ij} = \mathrm{rank}(b_{ij}Z)$	The rank increase achieved by adding the missing edge (i,j) using basis Z of $\ker B$.
Edge-Addition Independence	$\mathrm{add}^{ij}_{\mathrm{ind}} = \frac{\|b_{ij}Z\|_F}{\|b_{ij}\|_F}$	A metric in $[0,1]$ capturing how independently a missing edge (i,j) would contribute.
Edge-Addition Stiffness	$\Delta\lambda_{ij} \approx \|b_{ij}v\|^2$	The first-order approximation of how much adding an edge stiffens the weakest mode.
Edge-Removal Rank Loss	$\Delta\,\mathrm{rank}_{ij} = \mathrm{rank}(B) - \mathrm{rank}(B \setminus b_{ij})$	The loss in rigidity rank if an existing edge is removed.
Edge-Removal Conditioning Loss	$\mathrm{remove}^{ij}_{\mathrm{stiff}} = 1 - \frac{\lambda(B\setminus b_{ij})}{\lambda(B)}$	The fractional loss in conditioning quality if an existing edge is removed.
Block Leverage	$H = b(B^\top B)^+ b^\top$	A matrix whose unit eigenvalues indicate the directions constrained exclusively by a given edge.
Per-Measurement Sensitivity	$s_k = \|B_k^+\|_F^2$	The contribution of noise from a single measurement k to the total squared error.

Environment Design - Observation

$$o_t = \left[\underbrace{\mathcal{O}_{\text{node}}}_{\text{node features}} \;\middle|\; \underbrace{\mathcal{O}_{\text{pair}}}_{\text{pair features}} \;\middle|\; \underbrace{X}_{\text{positions}} \;\middle|\; \underbrace{A}_{\text{adjacency}} \;\middle|\; \underbrace{u}_{\text{action/pointer state}} \right]$$

$$\mathcal{O}_{\text{node}}(i) = \left[\mathbf{1}_{\text{dom}_i}, \frac{d_i^{\text{in}}}{m_{\text{req}}/n}, \frac{d_i^{\text{out}}}{m_{\text{req}}/n}, c_i^{\text{clo}}, c_i^{\text{eig}}, c_i^{\text{btw}}, \mathcal{O}_{\text{global}}, f_i, \|v_i^{\text{pos}}\|, \|v_i^{\text{att}}\| \right]$$

$$\mathcal{O}_{\text{pair}}(i,j) = \left[\beta_{ij}, A_{ij}, b_{ij}^{\text{btw}}, A_{ij}A_{ji}, \frac{(A^2)_{ij}}{m_{\text{req}}/n}, \underbrace{\text{add}_{\text{ind}}, \text{add}_{\text{rank}}, \text{add}_{\text{stiff}}}_{\text{adding } (i,j)}, \underbrace{\text{rem}_{\text{rank}}, \text{rem}_{\text{stiff}}}_{\text{removing } (i,j)} \right]$$

$$\mathcal{O}_{\text{global}} = \left[\frac{r_K - \text{rank}\, B}{r_K}, \frac{m}{m_{\text{req}}}, \mathbf{1}_{\text{IBR}}, q \right]$$

Quantity	Mathematical Expression	Description
Pairwise Quantities		
Add Independence	$\text{add}_{\text{ind}}^{ij} = \frac{\|b_{ij}Z\|_F}{\|b_{ij}\|_F}$	How independently the missing edge (i,j) contributes.
Add Rank	$\text{add}_{\text{rank}}^{ij} = \frac{\text{rank}(b_{ij}Z)}{c_{\max}}$	Normalized rank gain if edge (i,j) is added.
Add Stiffness	$\text{add}_{\text{stiff}}^{ij} = \frac{\|b_{ij}v\|_F}{\|b_{ij}\|_F}$	How much the edge stiffens the weakest mode.
Remove Rank	$\text{remove}_{\text{rank}}^{ij} = \frac{\text{rank}\, B - \text{rank}(B \backslash b_{ij})}{c_{\max}}$	Normalized rank loss if an existing edge is removed.
Remove Stiffness	$\text{remove}_{\text{stiff}}^{ij} = 1 - \frac{\lambda(B \backslash b_{ij})}{\lambda(B)}$	Conditioning loss if an existing edge is removed.
Node Quantities		
Position Softest Mode	v_i^{pos}	Position component of the softest mode at node i.
Attitude Softest Mode	v_i^{att}	Attitude component of the softest mode at node i.
Global Conditioning Quantity		
Conditioning Quality	$q = \text{sigmoid}\left(\frac{g - g_{\text{ref}}}{s}\right)$	The normalized conditioning-quality score, where $g \in \{\log_{10}\lambda, -\log_{10} a_{\text{opt}}, -d_{\text{opt}}\}$.

Environment Design - Reward Function

$$r_t = \underbrace{[\phi(s_{t+1}) - \phi(s_t)]}_{\text{improvement in formation quality}} - c_{\text{time}},$$

$$\phi(s) = \underbrace{w_r \frac{\operatorname{rank} B}{r_K}}_{\text{normalized rigidity}} - \underbrace{w_e \frac{m}{c_{\max}}}_{\text{normalized edge cost}} + \underbrace{\kappa \frac{w_e c_{\max}}{r_K} \mathbf{1}_{\text{IBR}}\, q}_{\text{conditioning bonus}},$$

$$q = \operatorname{sigmoid}\left(\frac{g - g_{\text{ref}}}{s}\right), \qquad g \in \{\log_{10}\lambda,\ -\log_{10} a_{\text{opt}},\ -d_{\text{opt}}\}.$$

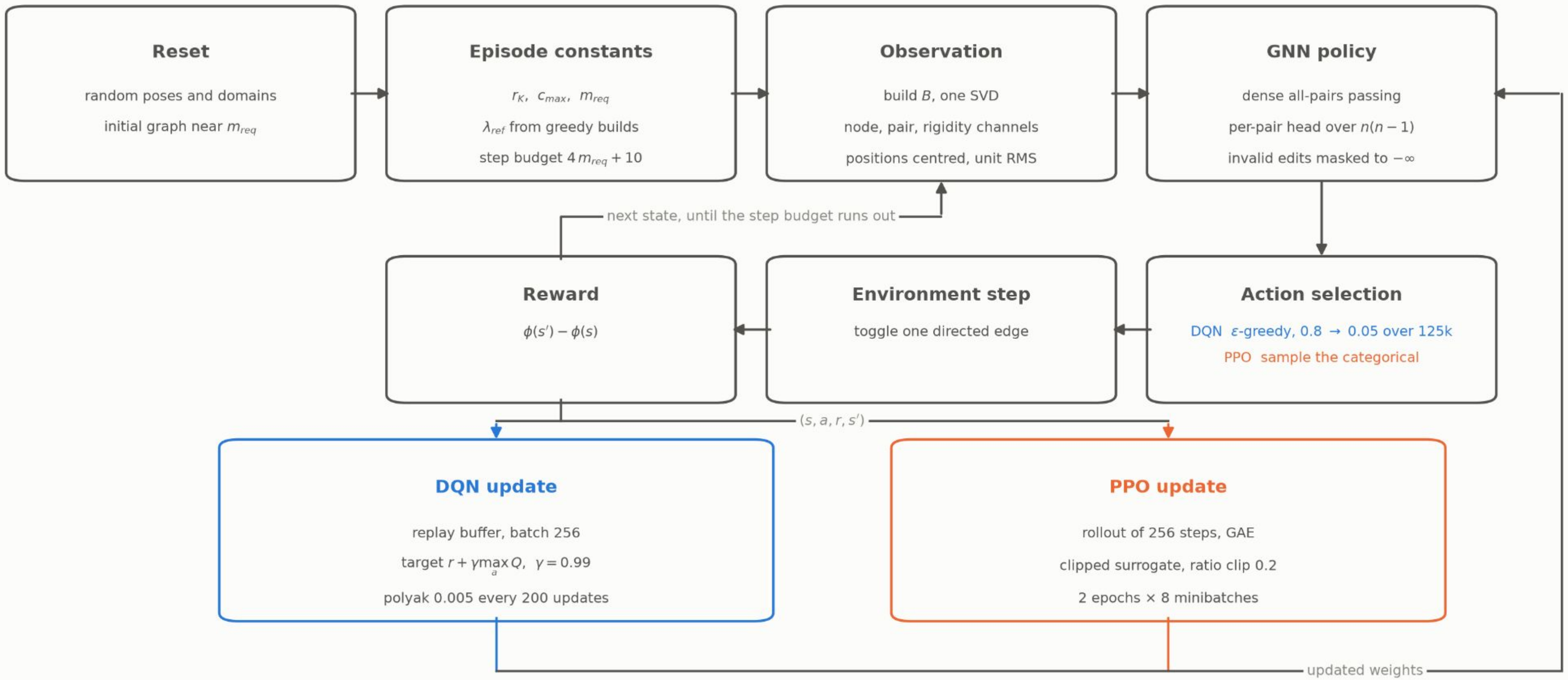
r_t	reward at time t,
s_t, s_{t+1}	current and next formation states,
c_{time}	time-step penalty,
B	extended bearing-rigidity matrix,
$\operatorname{rank} B$	current rigidity rank,
r_K	rank of the complete graph,
m	number of sensing edges,
$c_{\max}$	maximum number of edges that can be added,
w_r, w_e	weights on rigidity and edge cost,
$\mathbf{1}_{\text{IBR}}$	$= \begin{cases} 1, & \text{formation is infinitesimally bearing rigid (IBR)}, \\ 0, & \text{otherwise}, \end{cases}$
λ	smallest nonzero eigenvalue of $B^\top B$,
a_{opt}	$= \operatorname{tr}\big((B^\top B)^+\big)$ (A-optimal error criterion),
d_{opt}	$= -\sum_k \log w_k$ (D-optimal criterion),
g_{ref}	episode-specific reference value of g,
s	sigmoid width,
κ	trade-off weight for conditioning vs. sparsity.

$$\boxed{\text{Reward} = \text{change in normalized rigidity} - \text{edge/time cost} + \text{conditioning bonus}}$$

Training Scheme

Training scheme

One environment step edits one directed edge. The trunk is shared and only action selection and the update differ between the two algorithms.

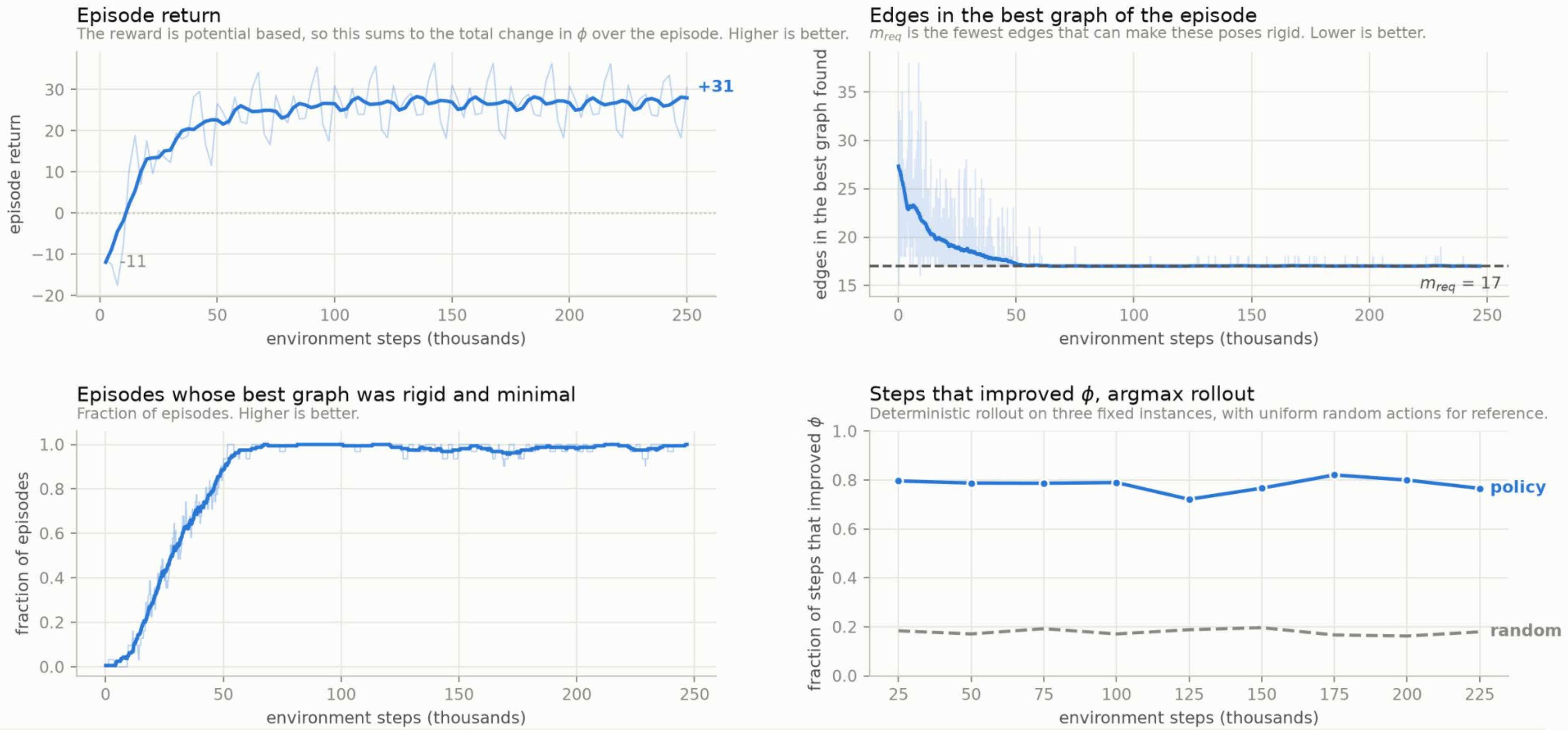


The episode constants depend on the poses and not on the edge set, so they are computed once at reset and reused by every step of the episode.
The reward is the change in ϕ rather than its level, and there is no stop action, so every step of the budget applies an edit.

Results

DQN training, n=10 mixed-domain formation

One directed edge added or removed per step, 250k environment steps. Thin line is per episode, thick line is a rolling mean.

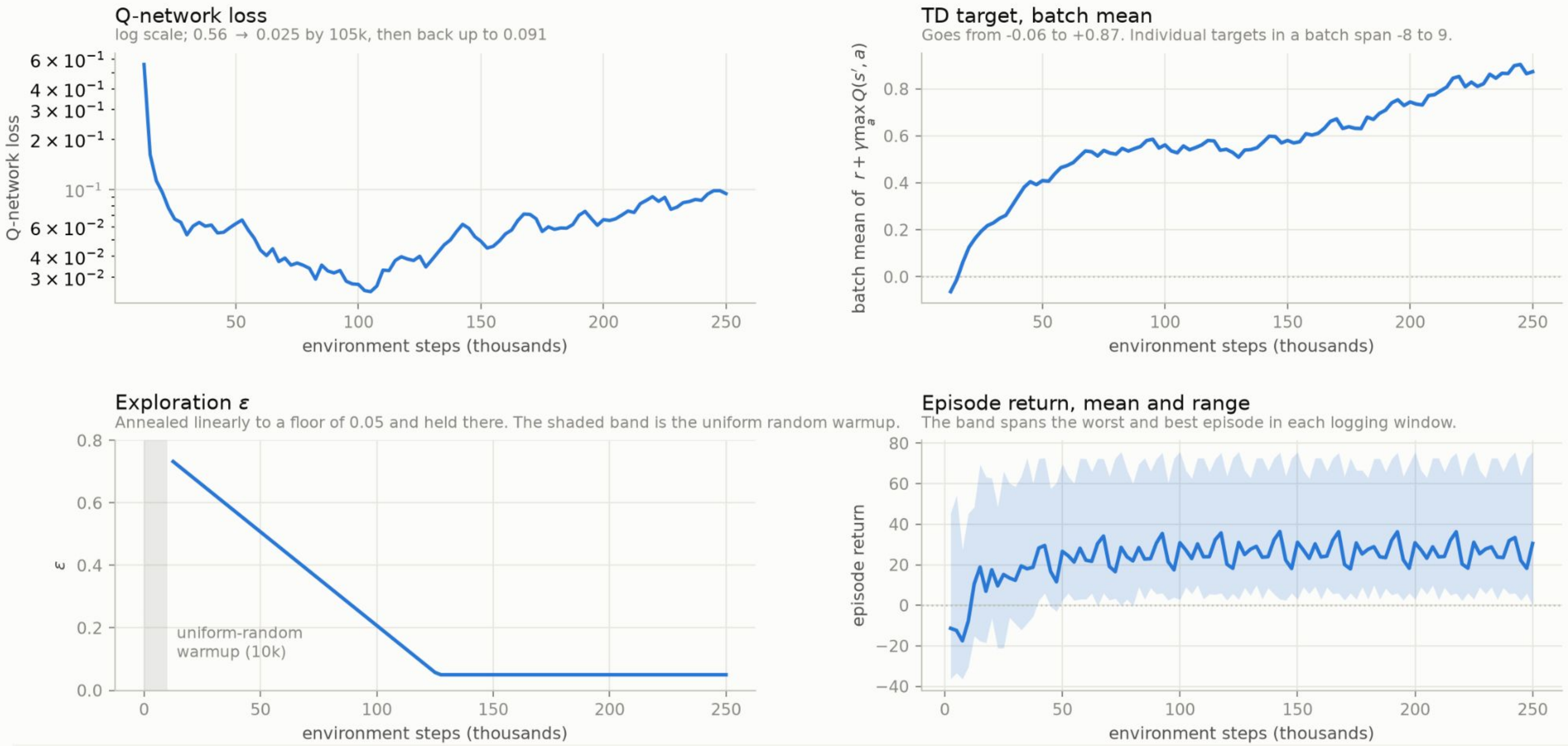


The first three panels are measured on the exploring policy during training. The fourth is a separate probe that takes the argmax action instead of sampling, on the same three held-out formations at every point, so the two are not directly comparable to each other.

Results

DQN update diagnostics

Quantities logged from inside the update over 250k environment steps. Batch size 256, target polyak 0.005 applied every 200 updates.

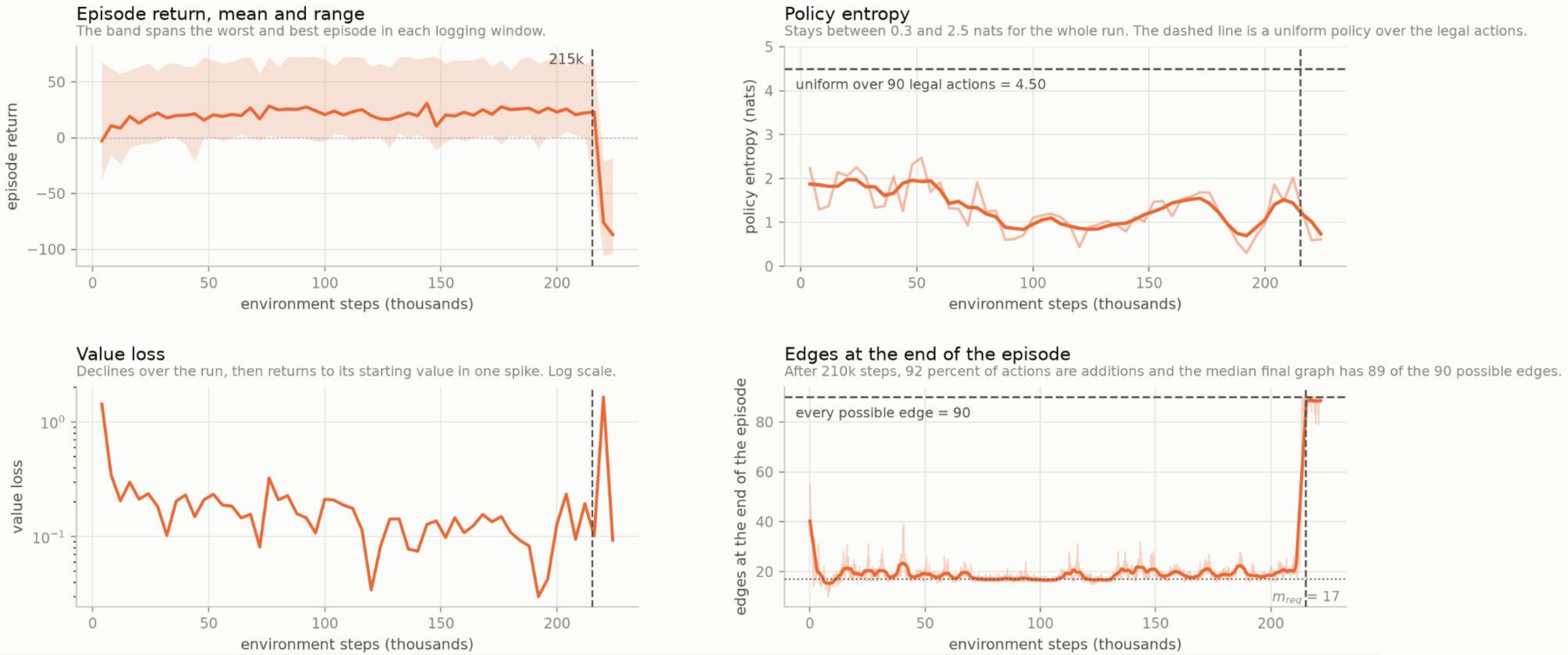


The loss falls for the first 100k steps and rises after that. The target moves with the network, every episode draws fresh poses, and an improving policy reaches states it has not fitted yet, so a rising loss beside a flat return is expected in this setup rather than a failure.

Results

PPO return collapse at 210k steps

Same run as the previous figure. There is no learning rate schedule and no exploration schedule, so nothing external changes at that point.

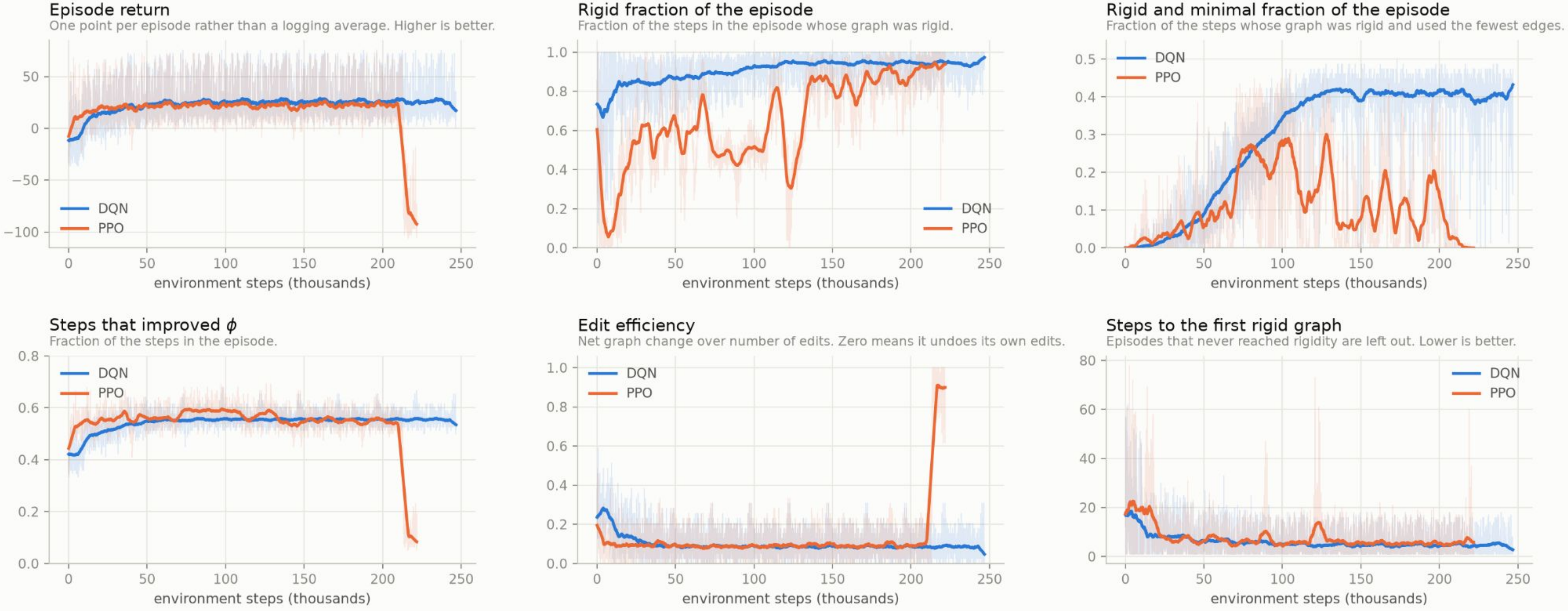


PPO trains only on trajectories from the current policy, so once the behaviour drifts the update contains nothing from the earlier region. DQN keeps those transitions in its replay buffer. Entropy stays inside its usual band across the collapse and the value loss moves only after the return has.

Results

Per-episode quantities during training, DQN and PPO

One point per finished episode, 3164 for DQN and 2843 for PPO, on the same n=10 task. Thin line is per episode, thick line is a rolling mean.

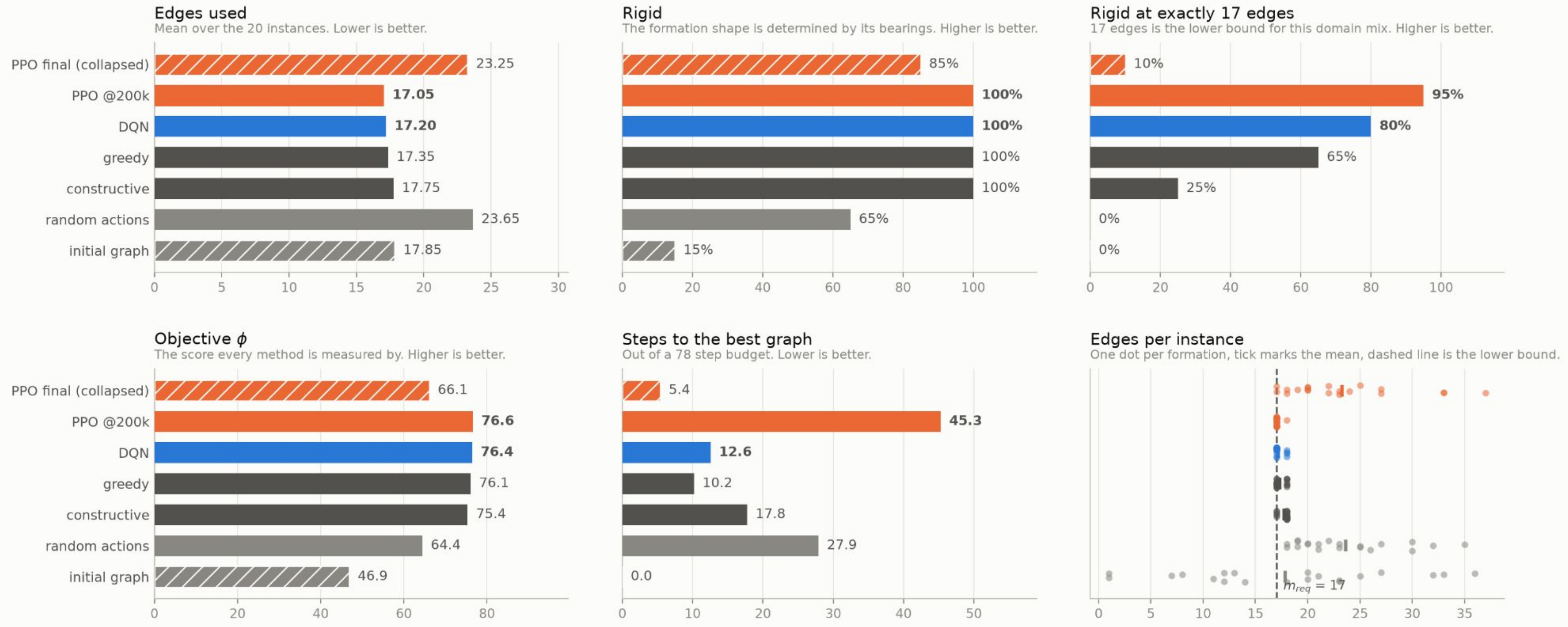


Both algorithms reach rigidity and both settle at an edit efficiency of 0.08, meaning they undo their own edits, which with no stop action is the only way to hold a graph. Over 100k to 200k steps DQN holds a minimum edge graph for 41 percent of an episode against 10 percent for PPO. Per-step decision quality is the same for both. 1.3 percent of PPO episodes never reach rigidity and are left out of the last panel.

Results

Evaluation against the classical baselines

20 held-out formations from a frozen benchmark, n=10 with five agent domains mixed. Same environment, instances and seed for every row.

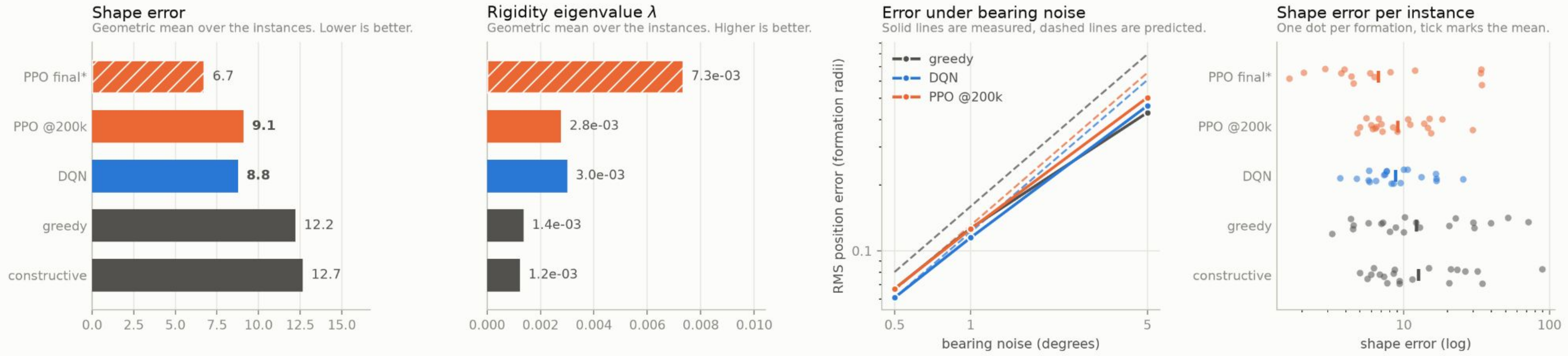


PPO at 200k uses 17.05 edges and reaches the lower bound on 19 of 20 formations, against 16 for DQN and 13 for greedy, but takes until step 45 of 78 to get there where DQN is done by step 13. The row above it is the same run 25k steps later, after the collapse: 23.25 edges and 2 of 20, which is what random actions score. The saved model file for that run is the collapsed one.

Results

Estimation quality on the evaluation benchmark

The same 20 held-out formations. Shape error is the RMS state error per radian of bearing noise, position in formation radii and attitude in radians.

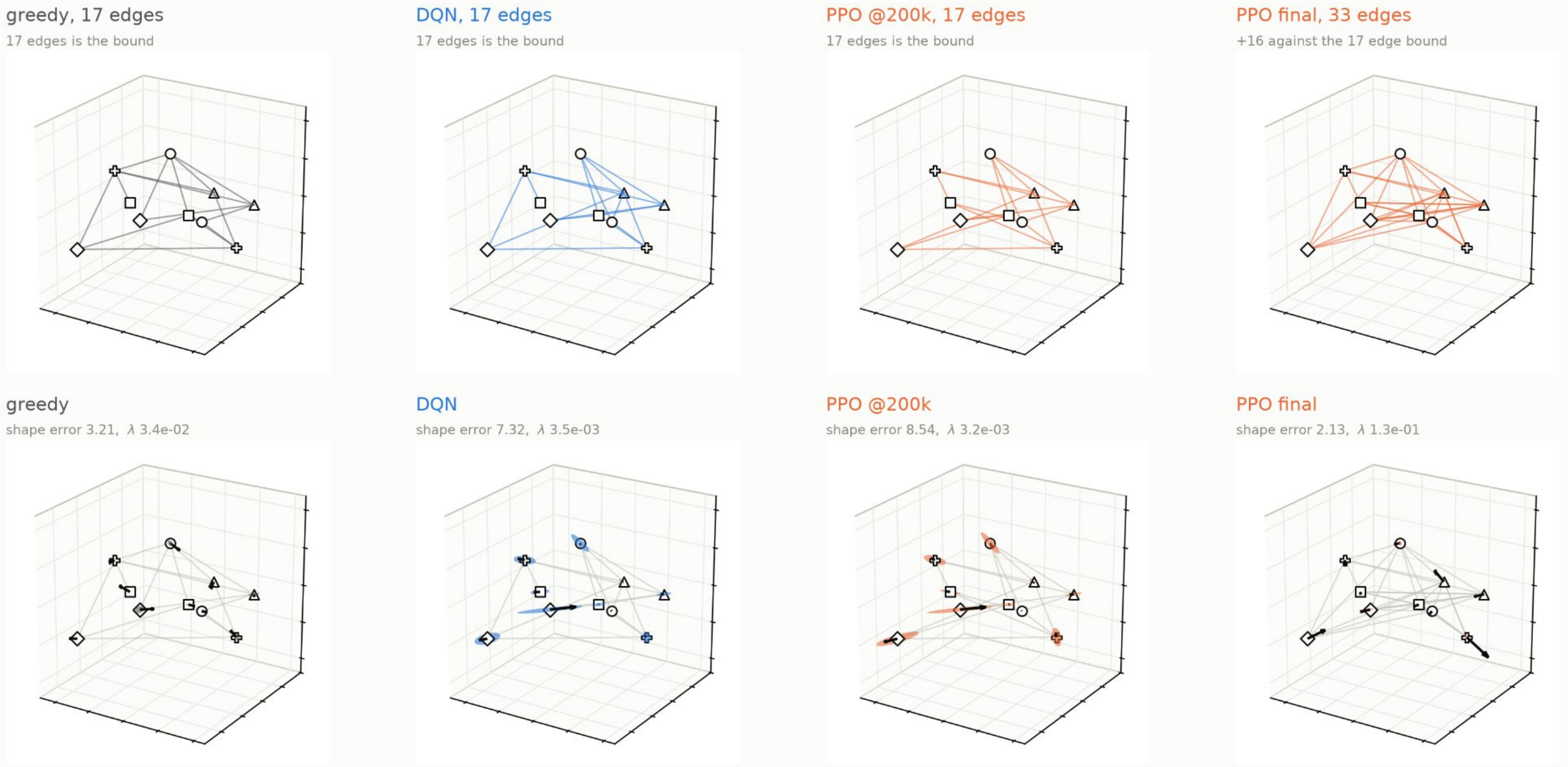


Shape error is a property of the graph and not only of its edge count, so two methods using the same number of edges can differ here.
The graphs themselves, and what they mean for the estimate, are in the topology figure. PPO final is rigid on 17 of 20 instances and its means cover only those, marked with an asterisk.

Results

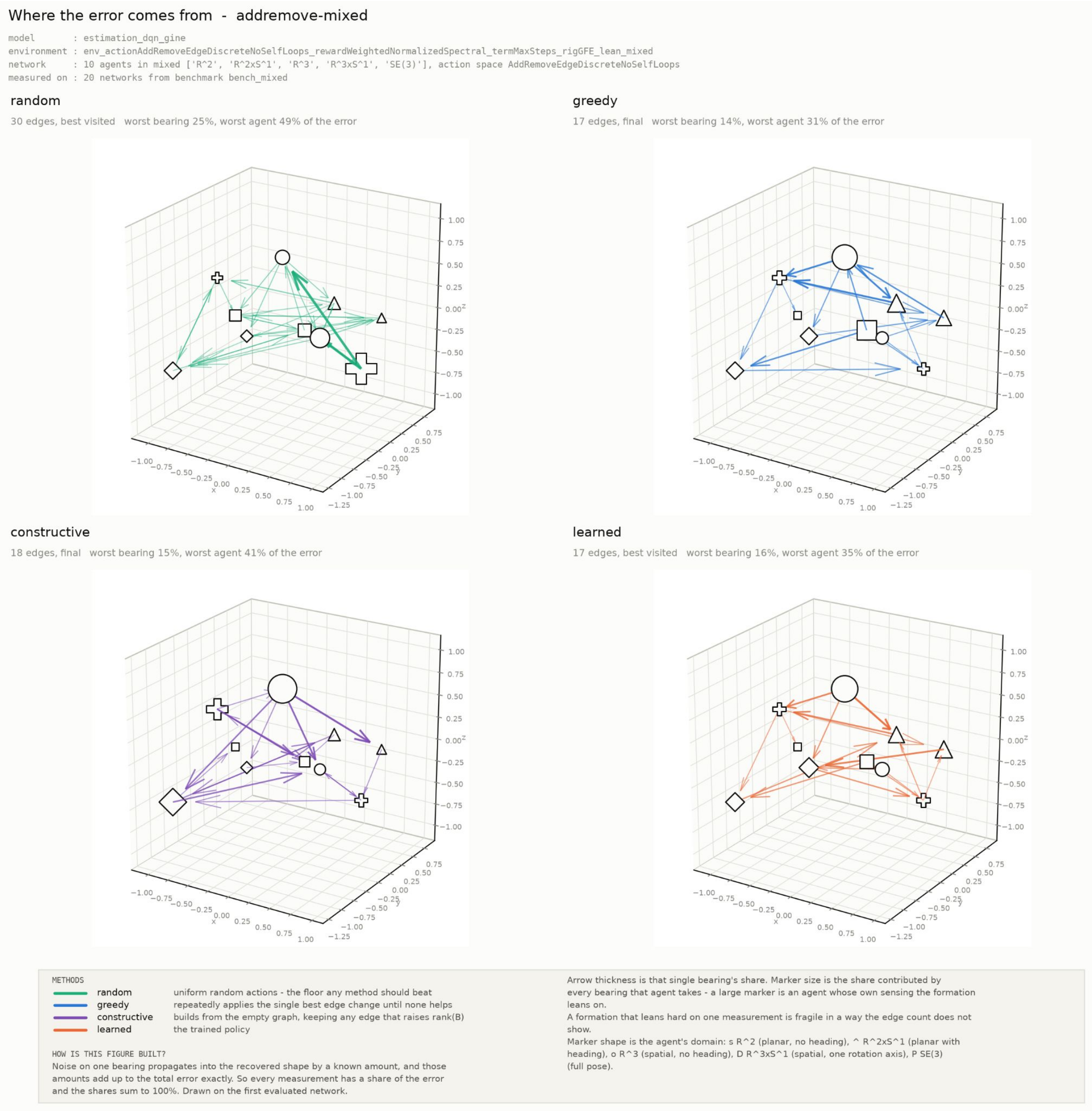
The graph each method built, and what it costs the estimate

The first benchmark formation, n=10. Agent marker is set by the agent domain.



Top row is the graph. Bottom row keeps the same graph in grey and adds the 1-sigma position uncertainty per agent at one radian of bearing noise, as a coloured blob, together with a black arrow along the softest mode, the deformation the bearings react to least. The blobs use one scale shared by all four panels so their sizes compare. Arrow lengths are scaled per panel and do not compare across panels. On this particular formation greedy is the better conditioned 17 edge graph, which is the opposite of the 20 instance average.

Results

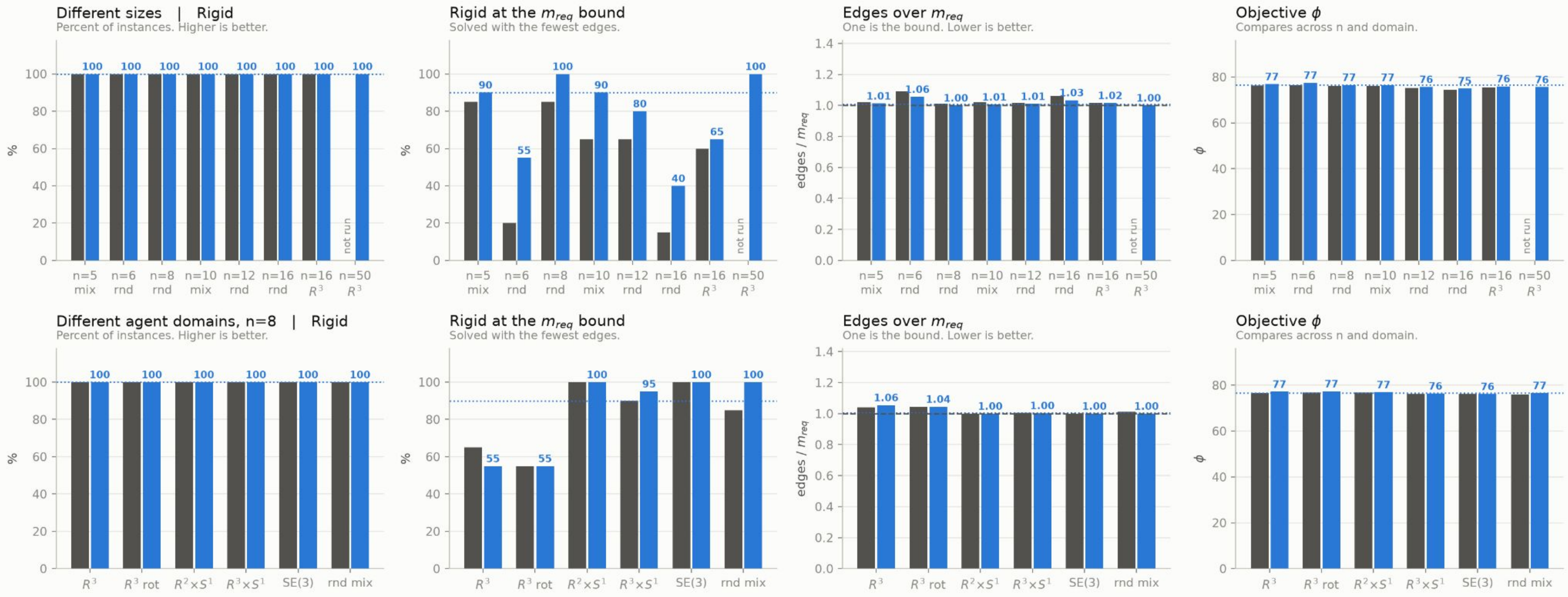

Where the error comes from - addremove-mixed
model : estimation_dqn_gine
environment : env_actionAddRemoveEdgeDiscreteNoSelfLoops_rewardWeightedNormalizedSpectral_termMaxSteps_rigGFE_lean_mixed
network : 10 agents in mixed ['R^2', 'R^2xS^1', 'R^3', 'R^3xS^1', 'SE(3)'], action space AddRemoveEdgeDiscreteNoSelfLoops
measured on : 20 networks from benchmark bench_mixed
random
30 edges, best visited worst bearing 25%, worst agent 49% of the error
greedy
17 edges, final worst bearing 14%, worst agent 31% of the error
constructive
18 edges, final worst bearing 15%, worst agent 41% of the error
learned
17 edges, best visited worst bearing 16%, worst agent 35% of the error
METHODS
random uniform random actions - the floor any method should beat
greedy repeatedly applies the single best edge change until none helps
constructive builds from the empty graph, keeping any edge that raises rank(B)
learned the trained policy
HOW IS THIS FIGURE BUILT?
Noise on one bearing propagates into the recovered shape by a known amount, and those amounts add up to the total error exactly. So every measurement has a share of the error and the shares sum to 100%. Drawn on the first evaluated network.
Arrow thickness is that single bearing's share. Marker size is the share contributed by every bearing that agent takes - a large marker is an agent whose own sensing the formation leans on.
A formation that leans hard on one measurement is fragile in a way the edge count does not show.
Marker shape is the agent's domain: s R^2 (planar, no heading), ^ R^2xS^1 (planar with heading), o R^3 (spatial, no heading), D R^3xS^1 (spatial, one rotation axis), P SE(3) (full pose).

Results

One policy across thirteen networks

greedy learned

estimation_dqn_gine was trained only on the mixed n=10 set. Every other column changes the size, the agent domains, or both. The dotted line is what it scores on its own training set.

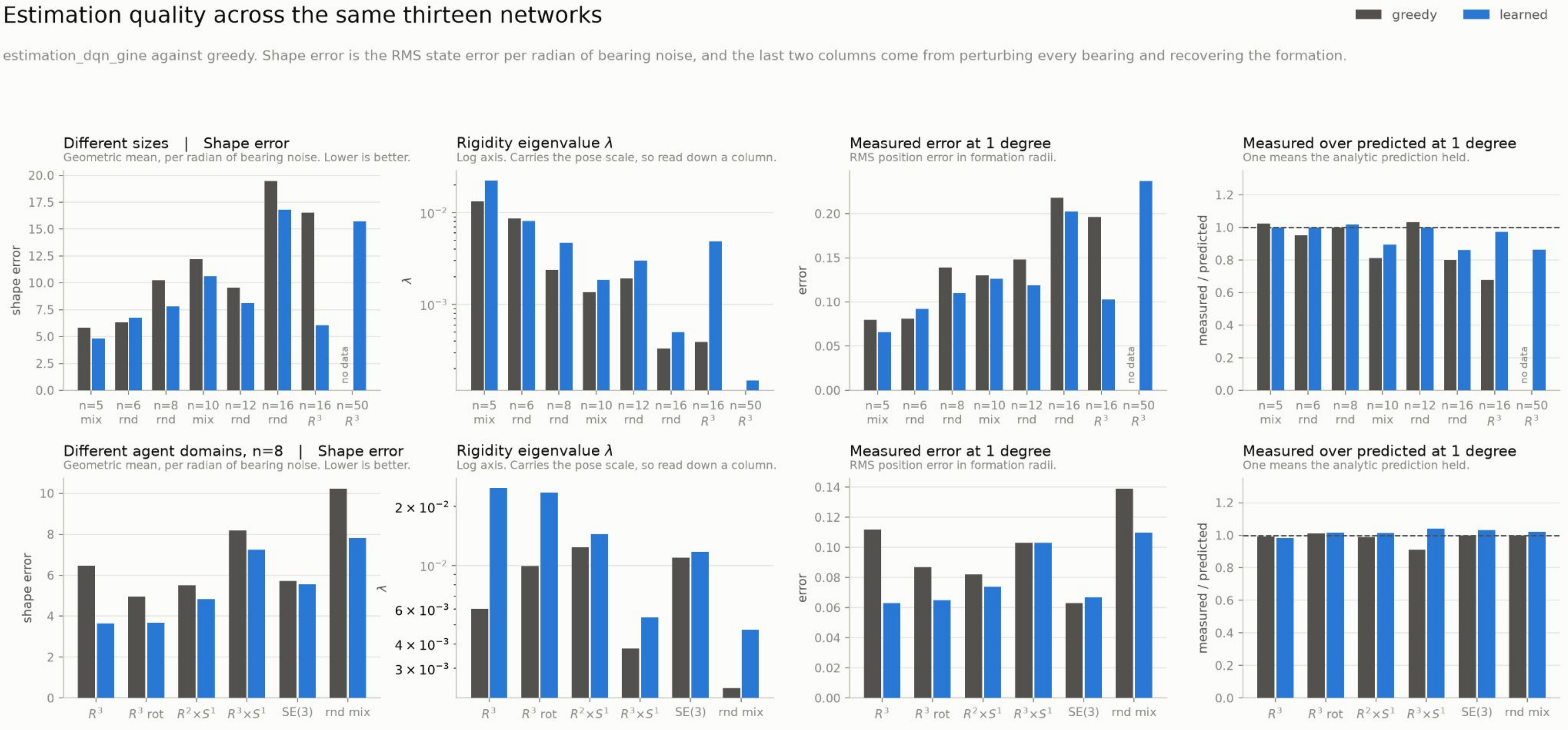


Top row varies the network size at a mixed or random domain draw, plus two homogeneous R^3 sets. Bottom row fixes n=8 and varies the domain, and includes the same R^3 poses under a global rotation, which the observation is not invariant to in the orientation-free domains.
Every set holds 20 instances except n=50, which holds 2, and greedy was not run on that one.

Results

Estimation quality across the same thirteen networks

estimation_dqn_gine against greedy. Shape error is the RMS state error per radian of bearing noise, and the last two columns come from perturbing every bearing and recovering the formation.

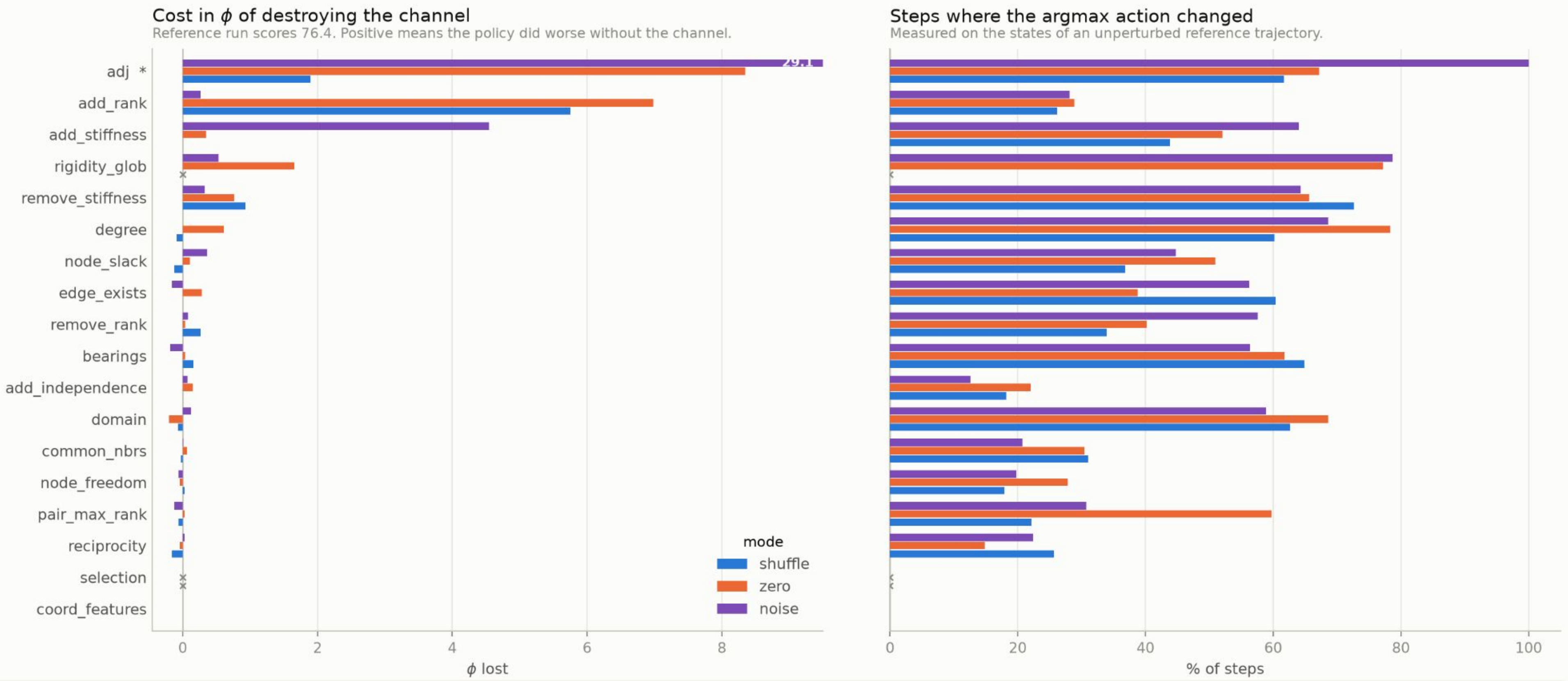


Shape error and the measured error are comparable across sets. The rigidity eigenvalue is not, since it carries the pose scale, so read it down a column rather than across.
The last column is the measured error divided by what the rigidity matrix predicts for the same graph. It falls below one where the noise is past the range the linear prediction covers.
Every set holds 20 instances except n=50, which holds 2, and greedy was not run on that one.

Results

Observation channel ablation

stiff_dqn_gine on 20 mixed n=10 instances. Each channel is destroyed on its own, in three ways, and the episode is re-run.



shuffle permutes the channel across nodes or pairs and keeps its distribution. zero and noise also change its scale, so a reaction there can be a response to an input the network never saw in training.
A cross marks a channel that mode could not perturb, which happens when the channel is constant along the shuffled axis. The starred channel feeds the action mask, so perturbing it changes which actions are legal rather than what the policy knows, and its bar is cut off at the axis with its value written in.

Conclusions

Summary
- Conducted mathematical analysis and derivations of bearing rigidity theory.
- The topology design problem was formulated as a Markov decision process over single edge edits.
- Proposed architectures and training schemes have been implemented.
- A frozen benchmark and a set of classical baselines were built to measure against.
- An estimation pipeline was added to check what a chosen topology costs under real bearing noise.
- The learned policy matches the classical baselines on edge count and reaches the theoretical minimum more often.
- Adding a conditioning term to the reward produces better conditioned graphs at the same edge count.
- Off-policy learning proved more stable than on-policy learning on this task.
- The policy converges early in the episode and then holds (oscillates), since the action space has no terminal action.
- Channel-wise ablation results indicate that the policy relies on rigidity-derived observations.

Limitations and Further Work
- Generalization of a single policy across agent domains is achieved, however scaling the network size is a strong limitation because of exploding computational complexity. Normalization of network-dependent values (e.g. spectral component in *phi*) depend on evaluating the greedy hill-climbing algorithm on the current graph, multiple times, which scales with the possible number of edges for a graph. Therefore, the required number of rigidity matrix construction and spectral decomposition operations has complexity $O(n^2)$. These operations themselves are costly in larger networks.
- Experiments with a stopping behavior (policy has a “stop” action) resulted in policies that constructed feasible graphs, however, no policy could outperform the heuristics in terms of robustness against measurement noise.
- The formulation is centralised, while the long-term goal is a distributed one.
- The informed and centralized policy may not be suited for deployment in real scenarios. A centralized training and decentralized execution (CTDE) approach, combined with an uninformed actor network, can be pursued as further research.